

Verifier-Guided Evaluation of LLM-Generated Network Configurations: a Paired Study with Shared Initial Candidates

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired confirmatory study (CAND-2026-001) that measured whether a verifier-triggered guarded pipeline (generate $\rightarrow$ validate $\rightarrow$ repair, ≤ 1 repair) reduces deterministic-invariant violations relative to single-shot initial generation for intent $\rightarrow$ configuration NetOps tasks. Using a deterministic synthetic task generator and a deterministic network verifier, we ran 200 preregistered tasks under shared_initial_candidate semantics with the hosted model gpt-5-mini (execution/model_configuration.json records temperature = null/unset). Baseline configurations passed the verifier in 199/200 tasks (99.5%); the guarded pipeline passed 200/200 (100%). Paired absolute risk difference (guarded - baseline) = 0.005; contingency counts n11=199, n10=1, n01=0, n00=0; exact two-sided McNemar $p = 1.0$; paired-bootstrap 95% percentile CI = [0.00, 0.015] (10,000 resamples, seed 1729). The single guarded improvement arose from one verifier-triggered repair that corrected a multi-step ordering violation. We report preregistration, full execution accounting (201 model calls: 200 shared initial + 1 repair), paired analysis, representative diagnostic artifacts, and bounded limitations grounded in the archived execution bundle.

I. INTRODUCTION

Generative large language models (LLMs) are increasingly proposed to translate high-level operator intent into device configurations for network operations (NetOps). Operational correctness for such workflows requires generated configurations to satisfy deterministic invariants (reachability, policy compliance, workflow ordering, and group-level safety). Prior prototype systems and benchmarks illustrate feasibility but leave open questions about how much deterministic verification and bounded repair alter acceptance rates when the identical initial sample is reused across comparison conditions.

This paper reports a preregistered paired experiment (CAND-2026-001) designed to isolate the marginal effect of a verifier-triggered automated repair on an identical generated candidate. Under shared_initial_candidate semantics the same single initial generation is deterministically reused by both baseline and guarded episodes; any paired difference therefore arises solely from the permitted validator-triggered repair. The primary estimand is the paired difference in per-task binary acceptance by a deterministic verifier (paired_success_rate_difference_guarded_minus_baseline). Because shared_initial_candidate semantics were enforced, this estimand measures a repair-only effect and does not capture potential benefits from independent guarded first-pass generations, constrained prompts, or multi-sample selection.

Contributions. (1) A preregistered, fully archived paired experiment executing 200 intent $\rightarrow$ configuration tasks with a deterministic verifier and a hosted LLM (gpt-5-mini) under shared_initial_candidate semantics. (2) Complete execution accounting and deterministic reconciliation showing 200 shared initial generations and 1 repair call (201 model calls total). (3) Artifact-grounded confirmatory analysis reporting baseline pass 199/200 and guarded pass 200/200 with contingency counts (n11=199, n10=1, n01=0, n00=0), paired risk difference 0.005, exact two-sided McNemar $p = 1.0$, and bootstrap 95% CI = [0.00, 0.015] (10,000 resamples, seed 1729). (4) A localized failure diagnosis for the single repaired pair using archived validator traces. (5) Detailed reproducibility guidance and bounded limitations tied to archived artifacts and reconciliation records.

II. RELATED WORK

Recent work on LLM-driven NetOps shows convergent interest in intent $\rightarrow$ configuration synthesis, verification-aware pipelines, and tool-augmented agent designs, but the supplied verified records reveal complementary strengths and limitations that motivated our paired design. NetConfEval and related intent $\rightarrow$ configuration prototypes demonstrate that LLMs can produce workable device configurations from natural-language intent and that task-style benchmarks are useful for early evaluation; these studies report feasibility and dataset-limited performance measures but do not uniformly integrate a deterministic verifier as the final oracle for acceptance decisions [1]. Deterministic configuration verification methods (e.g., formal and symbolic reachability/policy-check approaches) provide a clear operational oracle for syntactic and semantic invariants and have been developed in the NetOps literature; these verification techniques form the natural oracle used in our experiment and highlight why a verifier-centric evaluation is defensible for configuration-synthesis tasks [2].

Across adjacent domains, generate $\rightarrow$ validate $\rightarrow$ repair patterns have empirical support: test-in-the-loop program-repair and synthesis-with-tests pipelines show that validating generated artifacts against executable checks and iteratively repairing failures materially improves correctness in software-engineering tasks [4]. Those demonstrations motivate guarded NetOps pipelines but are not direct evidence of net effects in network-configuration contexts because NetOps invariants and multi-device sequencing introduce distinct dependency

patterns. Tool-augmented agent and model-context approaches further extend capability by granting controlled access to specialized solvers, validators, or execution environments; evaluations of such tool-augmented designs argue they enable more complex, verifiable behavior but also point out engineering and interoperability challenges when integrating heterogeneous oracles [6].

Survey and reporting-guideline literature places additional context around rigor and reproducibility: broad LLM surveys synthesize model capabilities and limitations at scale and underscore the importance of careful experimental reporting and preregistration when evaluating generative systems [5]. Reporting guidelines for studies that use LLMs emphasize explicit documentation of sampling, validation, and analysis choices to avoid overclaiming—this influenced our preregistration and artifact-archiving decisions [3]. Finally, focused work on hallucination detection and validator gating in generative systems highlights common failure modes (confabulation, ordering errors, and adversarially induced fabrications) and motivates verifier gates as necessary but not sufficient mitigations; such analyses support our guarded pipeline as a targeted, bounded intervention rather than a general cure [7].

Synthesis. The supplied and verified literature collectively supports three points: LLM-based intent→config pipelines are feasible and actively prototyped; generate→validate→repair is a promising architectural pattern whose empirical demonstrations to date are stronger in code and testable artifacts than in NetOps-specific verifier studies; and rigorous reporting and verifier-grounded evaluations are necessary to interpret operational claims. Our paper addresses a narrower, previously underexplored empirical gap in the supplied records: a reproducible paired measurement of the marginal (repair-only) effect when the identical initial candidate is reused across baseline and guarded conditions. By enforcing shared_initial_candidate semantics and archiving the verifier traces, the present study isolates that repair-only estimand and complements prior prototype and generate–validate–repair demonstrations [1],[2],[4],[6],[3],[5].

III. METHODOLOGY

Preregistration and design freeze. We preregistered study CAND-2026-001 and froze the confirmatory design prior to model calls. The preregistration (preregistration_sha256 recorded in the execution manifest) fixed the primary estimand, the paired design, the selection rule for the deterministic task sample, generation semantics, maximum repair calls, the primary confirmatory test, and the missingness/treatment rules. The study_id is CAND-2026-001 and the execution contract declared generation_semantics = shared_initial_candidate, initial_generation_calls_per_task = 1, and maximum_repair_calls_per_task = 1.

Deterministic task selection and manifest encoding. The 200 tasks were selected deterministically from a synthetic task bank produced by deterministic_netops_task_generator_v5 according to the preregistered index rule (for zero-based ordinal j in $0..199$: $\text{cycle} = j // 5$; $\text{offset} = [4,5,6,7,8][j$

$\% 5]$; $\text{task_index} = 8 * \text{cycle} + \text{offset}$). Each task manifest (execution/task_manifest.jsonl) encodes: initial_state, expected_final_state, required_sequence (an ordered command list that the verifier enforces), allowed_touched_fields, workflow_policies (e.g., group constraints and minimum-active requirements), difficulty metadata (changed_interface_count, dependency_rule_count, required_command_count), and a normalized reference_answer. Representative tasks in the archive show required_command_count $\in \{3, \dots, 9\}$ and difficulty labels very_hard or extreme; these manifest fields are authoritative inputs to the deterministic_netops_validator_v5 scoring logic.

Model configuration and sampling semantics. The requested hosted model is declared as gpt-5-mini in execution/model_configuration.json. That artifact records provider = openai_responses, declared_model_version = gpt-5-mini, max_output_tokens = 1500, maximum_attempts_per_call = 1, reasoning_effort = "minimal", store = false, and temperature = null (unset). We explicitly note that temperature = null/unset does not imply deterministic sampling or temperature = 0; nondeterministic hosted-model sampling may still occur. The preregistered adapter-level contract enforces shared_initial_candidate semantics: a single initial generation call per task was deterministically reused by both baseline and guarded episodes (no independent per-condition sampling). The run’s deterministic reconciliation (archived) reports stage_counts = {shared_initial_generation: 200, repair: 1} and hosted_model_call_event_count = 201, confirming the planned call pattern.

Sanitization, exclusions, and missingness handling. The preregistration specified an automatic syntax sanitizer (automatic_syntax_sanitizer_v1) to deterministically correct minor formatting issues prior to verifier parsing. Exclusion rules allowed deterministic pre-call exclusion only when a vendor-template token-normalized match indicated potential leakage or if the provided input failed syntactic pre-checks; such deterministic exclusions would be logged. The missingness_plan declared primary failed-call handling (complete_pair_primary) and a sensitivity treatment that would treat persistent unparseables as failures. In the archived run no preregistered exclusions occurred and there were no persistent unparseables; missingness artifacts (analysis/missingness_summary.csv) record complete_pairs = 200 with zeros elsewhere.

Deterministic verifier, scoring rule, and validator traces. The oracle was deterministic_netops_validator_v5 (validator_version 5.0); it implements a deterministic parser and an invariant checker that enforces per-task constraints encoded in each task manifest. The verifier’s full_intent_constraint_validation returns 1 only when: (a) all required_sequence ordering constraints are satisfied, (b) allowed_touched_fields are respected, (c) group-level invariants (final_group_constraints, minimum_active_in_groups) hold, and (d) no protected interfaces or preserved fields are modified. Per-episode validator_trace artifacts (archived under execution/scoring/) include validation_before

and validation_after objects, final_configuration, normalized_configuration, parsed_command_count, required_command_count, observed_touched_field_count, violation_count, violation_codes, validator_id, and validation boolean valid. These traces are the authoritative evidence for per-episode scores and repair decisions.

Paired execution semantics and repair mechanism. Under shared_initial_candidate semantics the identical sampled candidate (shared_initial_candidate) is provided to the baseline and guarded episodes. The guarded pipeline permits at most one automatic repair call triggered when the verifier reports violations; the repair is implemented as a single repair prompt to the hosted model (repair stage_count = 1 observed). Because baseline and guarded reuse the same initial candidate, any paired success difference can only arise from the permitted validator-triggered repair. This design isolates the repair-only estimand and does not measure potential benefits from independent guarded-first-pass generations or constrained prompting variants.

Statistical analysis plan and exact-test implementation. The preregistered primary test for the binary paired outcome (all-specified-invariants-pass) is the exact two-sided McNemar test computed from discordant pair counts n10 (guarded=1, baseline=0) and n01 (guarded=0, baseline=1). We implement the exact two-sided McNemar via the exact binomial tail: for $n = n10 + n01$ and $k = n10$, compute $p = 2 * \min\{\text{BinomCDF}(k; n, 0.5), 1 - \text{BinomCDF}(k - 1; n, 0.5)\}$ with conventional handling for $k = 0$. Paired-bootstrap percentile confidence intervals for the absolute risk difference (guarded - baseline) were planned as an auxiliary estimator; archived analysis used bootstrap_resamples = 10000 and bootstrap_seed = 1729 (parameters recorded in analysis/analysis_log.jsonl). The primary inference uses complete_pair_primary handling; the preregistered sensitivity treating persistent unparsables as failures is supported by the archived artifacts but was not needed for this run.

Reproducibility and artifact linkage. The preregistration, model_configuration.json, task manifests, raw model responses (execution/responses/), per-episode scoring artifacts (execution/scoring/), the deterministic reconciliation (analysis/deterministic_reconciliation.json), and the analysis artifacts (analysis/*.csv, analysis/analysis_log.jsonl) together provide the machine-verifiable evidence trace for every methodological choice described above. The initial master prompt is archived (provenance/master_prompt.txt; SHA-256 recorded) and the execution manifest lists representative artifact hashes; these archived items are the authoritative sources for the methodological parameters reported here.

IV. RESULTS

Execution accounting and provider audit. The archived execution manifest records start and end timestamps (UTC) showing the run elapsed ≈ 29 minutes (start 2026-09-01T16:03:05.491039+00:00, end 2026-09-01T16:32:13.298536+00:00). Planned_episode_count = 400 and completed_episode_count = 400; failed_episode_count =

0. Provider-call accounting (deterministic_reconciliation.json) reports hosted_model_call_event_count = 201 with stage_counts = {shared_initial_generation: 200, repair: 1}. Cache_miss_count = 201 and cache_hit_count = 0 indicate fresh model generations for all shared-initial calls and a single repair-stage call; these counts match the preregistered shared_initial_candidate semantics and the recorded single repair invocation.

Primary confirmatory outcomes. analysis/condition_summary.csv shows baseline successes = 199 and failures = 1 (baseline success_rate = 0.995) and guarded successes = 200 and failures = 0 (guarded success_rate = 1.000). The paired contingency table (analysis/paired_contingency_table.csv) records n11 = 199, n10 = 1, n01 = 0, n00 = 0. From these counts the paired absolute risk difference (guarded - baseline) = 0.005. Using the exact two-sided McNemar formulation described in Methods, with $n = n10 + n01 = 1$ and $k = n10 = 1$ we compute $\text{BinomCDF}(1; 1, 0.5) = 1$ and $1 - \text{BinomCDF}(0; 1, 0.5) = 0.5$; hence $p = 2 * \min(1, 0.5) = 1.0$. The paired-bootstrap percentile 95% confidence interval for the absolute risk difference was computed with 10,000 resamples and bootstrap_seed = 1729 (analysis/analysis_log.jsonl) and equals [0.00, 0.015]. These numeric artifacts (analysis/condition_summary.csv, analysis/paired_contingency_table.csv, analysis/analysis_log.jsonl) are archived and reproduce the reported confirmatory values when the paired_binary_analysis_v1 executor is re-run on the archived input_results_sha256.

Single repair event and diagnostic traces. The guarded pipeline invoked exactly one automated repair. Archived validator traces for the repaired pair indicate an extreme composite task (difficulty metadata recorded in the task manifest: required_command_count = 9, changed_interface_count = 3, difficulty.level = "extreme") whose shared initial candidate violated the task's required_sequence ordering constraint. The verifier trace for that episode documents validation_before.violation_codes showing the ordering breach, the shared_initial_candidate (the identical candidate reused by baseline and guarded), and validation_after with validation_after.valid = true once the bounded repair was applied. The repair prompt/trace and the validator's normalized_configuration show the localized reordering the repair enacted; after repair the scorer returned score = 1 (scoring_reason_code = "VALID_CONFIGURATION"). These per-episode fields are present in the execution/scoring/* artifacts and constitute the archived evidence that the single observed paired improvement resulted from a verifier-triggered reordering repair that converted a sequenced-dependency failure into a valid configuration under the verifier's invariants.

Missingness, exclusions, and contamination checks. The run recorded no preregistered exclusions and no persistent unparsables; analysis/missingness_summary.csv reports complete_pairs = 200 and zeros for all other missingness categories. analysis/contamination_summary.csv is empty (no flagged contamination). Deterministic_reconciliation.json

indicates `pair_accounting_consistent = true` and `all_deterministic_consistency_checks_passed = true`. No episodes have `terminal_error_type` set, and provider-call audit details align with preregistration (200 shared initial generations, 1 repair). These artifacts support the statement that the confirmatory analysis included the full preregistered sample without post-hoc exclusions.

Exploratory diagnostics by difficulty metadata. Representative task manifests archived in `execution/task_manifest.jsonl` encode difficulty metadata (`changed_interface_count`, `dependency_rule_count`, `required_command_count`). In the archived representative subset `required_command_count` values span 3–9 and `changed_interface_count` values up to 3; the lone baseline failure is associated with the extreme composite difficulty pattern (highest `required_command_count` in the sample), consistent with the preregistered exploratory hypothesis that multi-device sequencing and dependency complexity concentrate residual failures. `analysis/paired_difference_figure.svg` (archived) visualizes condition rates and the bootstrap distribution used for the CI; those visual diagnostics and the per-task `validator_trace` fields (`parsed_command_count`, `observed_touched_field_count`, `violation_codes`) are available for targeted reanalysis to inspect whether other high-difficulty tasks produced near-miss traces that would benefit from alternative repair strategies.

Synthesis of quantitative uncertainty. The realized baseline success rate ($199/200 = 99.5\%$) places the confirmatory test in a sparse-discordant regime: with only one discordant pair the exact McNemar p-value gives limited directional evidence ($p = 1.0$), while the paired-bootstrap CI $[0.00, 0.015]$ quantifies the upper bound of plausible absolute improvement attributable to repair in this repair-only estimand. In practical terms, these results show that, for this synthetic verifier-backed task bank and the single-model run, bounded verifier-triggered repair corrected one expressible sequencing fault; they do not imply large marginal gains across broader settings without further experiments (e.g., independent guarded-generation contrasts, multi-model studies, or live dataplane emulation).

V. DISCUSSION

Interpretation under shared `initial_candidate` semantics. Because baseline and guarded conditions reuse the identical sampled candidate per pair, observed differences isolate the marginal effect of permitting an automated verifier-triggered repair. The experiment therefore quantifies a repair-only effect and does not evaluate other guarded variants that would permit independent guarded first-pass generations or constrained prompting. We reiterate this estimand interpretation explicitly so readers do not generalize to independent-generation contrasts without new experiments.

Statistical regime, power, and limitations of inference. Our preregistered power rationale targeted detection of moderate absolute paired increases ($\sim 10\text{--}15$ percentage points) at $N = 200$ under mid-range baseline assumptions ($\sim 45\text{--}65\%$). The realized baseline success rate ($199/200 = 99.5\%$) places the analysis in a sparse-discordant regime where the exact

McNemar p-value gives limited directional evidence ($p = 1.0$ for our observed discordants). The paired-bootstrap CI $[0.00, 0.015]$ provides a bounded estimate of plausible effect sizes compatible with the data and shows the dataset is consistent with at most small absolute improvements under our repair-only design. This underscores that unexpectedly high baseline performance can substantially reduce realized inferential sensitivity for small marginal effects. Practically, when baseline acceptance is already near one, corrective repairs can still be operationally useful for rare but high-consequence failures; however, the present data quantify only that such rare corrections occurred at least once in the preregistered sample and do not permit precise frequentist inference about very small effect magnitudes beyond the bootstrap bounds reported.

Mechanistic interpretation of the single repair. The archived `validator_trace` shows the repaired case involved a multi-step ordering violation across multiple interfaces and `required_command_count = 9`. The repair used by the guarded pipeline enacted a localized reordering consistent with the task manifest’s `required_sequence` and produced `validation_after.valid = true`. This pattern aligns with our exploratory hypothesis (H3) that multi-device sequencing and inter-device dependency rules concentrate residual failures. From a mechanistic standpoint, verifier-encoded ordering constraints are a concretely checkable invariant; when the initial candidate contains an expressible sequence breach, a bounded repair that reorders commands according to manifest constraints can correct the fault. That corrective potential depends directly on (a) the verifier’s ability to detect an expressible invariant breach and (b) the repair routine’s capacity to deterministically construct a corrected sequence without introducing new invariant violations.

Operational implications and bounded claims. Artifact-grounded evidence supports a narrow operational conclusion: in this synthetic deterministic task bank using `gpt-5-mini` and `deterministic_netops_validator_v5`, a verifier-triggered bounded repair corrected a localized multi-step ordering fault in one extreme task. This is direct evidence that verifier-guided bounded repair can remedy certain ordering/dependency violations in generated configurations when those violations are expressible in the task manifest’s invariants. It is not evidence that bounded repair will correct omissions only observable at dataplane runtime, or that guarded repair will address adversarially poisoned contexts; those remain open evaluation axes requiring live emulation, multi-model studies, and adversarial contamination experiments. Moreover, because our repair allowance was limited to a single automatic repair call per task, the observed effect quantifies what a minimally permissive guarded pipeline can accomplish; pipelines that permit multiple repair iterations, richer external-tool synthesis, or human-in-the-loop adjudication may achieve larger gains but were not evaluated here.

Threats to validity and generalization (expanded). Internal validity: the paired shared `initial_candidate` design isolates repair effects but does not address prompt sensitivity, model sampling variability, or the potential for different

initial candidates under guarded prompting strategies. The `model_configuration.json` records `temperature = null (unset)`, which is not evidence of deterministic sampling; nondeterminism in hosted-model generation remains a background condition that the `shared_initial_candidate` semantics mitigated for the paired comparison but did not eliminate as a systemic source of variance. Construct validity: the verifier enforces encoded syntactic and ordering invariants but cannot capture behaviors observable only through dataplane testing (e.g., runtime routing convergence or vendor-specific command side-effects). External validity: the experiment used a single model (gpt-5-mini) and a synthetic, deterministic task bank; results do not imply cross-model or production-device generality. Conclusion validity: with a single discordant pair the realized inferential leverage for small repair-only effects is weak; the bootstrap CI quantifies that uncertainty and highlights the value of larger samples or targeted sampling focusing on high-difficulty strata for future studies.

Practical considerations for guarded pipeline design. The findings articulate two practical design lessons grounded in archived artifacts: (1) deterministic verifiers that explicitly encode ordering and group-level invariants can serve as effective selectors for when automated repair is necessary; the verifier’s binary valid flag cleanly triggers bounded repair without human intervention in our run. (2) When invariants are expressible and repairs are constrained (for example, limited to reordering or restricted field modifications), a single focused repair may suffice to recover validity for complex tasks. Both lessons depend on careful manifest design that encodes the relevant invariants and on repair primitives that are conservative (minimize unintended changes) and auditable.

Reproducibility and next evaluation axes. The archived execution and analysis artifacts provide a complete trace for re-analysis and extension: task manifests with `required_sequence` encodings, per-episode `validator_trace` outputs, model-call accounting, and bootstrap parameters (seed = 1729, resamples = 10000). To better estimate small repair-only effects or to evaluate alternative guarded semantics, subsequent preregistered runs should consider (a) stratified oversampling of extreme/high-complexity tasks to increase discordant counts, (b) multi-model comparisons, and (c) relaxing `shared_initial_candidate` semantics to measure independent guarded first-pass benefits. Each such axis requires careful preregistration to avoid post-hoc selection and to preserve interpretability of paired or unpaired estimands.

VI. LIMITATIONS

- Synthetic deterministic task bank and verifier: results reflect correctness under the chosen synthetic intent templates and deterministic verifier and do not guarantee similar performance on live heterogeneous production devices or telemetry.
- Single-model experiment (gpt-5-mini): findings do not demonstrate multi-model generality.
- `Shared_initial_candidate` semantics: baseline and guarded conditions began from the same initial generation; the

study measures verifier/repair effects only and does not evaluate independent first-pass generation contrasts.

- Limited adversarial/contamination testing: the run did not include systematic adversarial retrieval or poisoned-context attacks; such threats remain an open evaluation axis.
- Oracle coverage: the deterministic verifier enforces encoded invariants but may omit runtime behaviors and device-specific semantics observable only through live execution; external validity to live deployments is therefore limited.

VII. CONCLUSION

We executed a preregistered paired experiment measuring the marginal effect of a verifier-triggered repair under `shared_initial_candidate` semantics for LLM-generated network configurations. In 200 synthetic deterministic tasks with gpt-5-mini and `deterministic_netops_validator_v5`, baseline acceptance was 199/200 and guarded acceptance was 200/200; a single automated repair corrected a multi-device ordering violation. Paired absolute risk difference = 0.005 with paired-bootstrap 95% CI [0.00, 0.015] (10,000 resamples, seed 1729) and exact two-sided McNemar $p = 1.0$. Artifact-grounded evidence therefore supports a constrained conclusion: verifier-guarded bounded repair can correct localized ordering faults in this synthetic verifier-backed setting. Broader operational claims require additional experiments: multi-model studies, independent-first-pass guarded semantics, adversarial contamination testing, and live-device or dataplane emulation to capture runtime semantics not modeled by the deterministic verifier.

DISCLOSURE STATEMENT

Disclosure: This run implemented preregistered study CAND-2026-001 and was executed autonomously after the autonomy lock. Declared model: gpt-5-mini (execution/`model_configuration.json` records `temperature = null/unset`; `null/unset` is not evidence of deterministic sampling or `temperature = 0`). Orchestration adapter: `hosted_netops_gvr_v1`. Exact initial master prompt archived at `provenance/master_prompt.txt` (SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Preregistration fixed `generation_semantics = shared_initial_candidate` (one sampled initial generation per task was deterministically reused by baseline and guarded episodes) and allowed at most one automated repair call per task. Model-call accounting in the archived execution manifest reflects 200 shared initial generation calls and 1 repair call (201 model calls total). The deterministic verifier used was `deterministic_netops_validator_v5`. Humans selected the broad topic family and constrained the initial master prompt prior to the autonomy lock as permitted by the track; no human manually edited technical manuscript content after the autonomy lock. All provider traces, verifier logs, task manifests, model-configuration records, and analysis artifacts

are archived in the execution bundle and indexed by the execution manifest.

REFERENCES

- [1] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, Marco Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?", 2024, 10.1145/3656296
- [2] Ryan Beckett, Aarti Gupta, Ratul Mahajan, David Walker, "A General Approach to Network Configuration Verification", 2017, 10.1145/3098822.3098834
- [3] Jack Gallifant, Majid Afshar, Saleem Ameen, Yindalon Aphinyanaphongs, Shan Chen, Giovanni Cacciamani, Dina Demner-Fushman, Dmitriy Dligach, Roxana Daneshjou, Chrystinne Oliveira Fernandes, Lasse Hyldig Hansen, Adam Landman, Lisa Soleymani Lehmann, Liam G. McCoy, Timothy A. Miller, Amy C. Moreno, Nikolaj Munch, David Restrepo, Guergana Savova, Renato Umeton, Judy Wawira Gichoya, Professor Gary S. Collins, Karel G.M. Moons, Leo Anthony Celi, Danielle S. Bitterman, "The TRIPOD-LLM reporting guideline for studies using large language models", 2025, 10.1038/s41591-024-03425-5
- [4] Yunhe Li, "Test-in-the-loop LLM Repair: Verifiable Automated Program Repair on QuixBugs with a "Failing Test $\rightarrow$ Patch $\rightarrow$ Regression Test" Loop", 2024, 10.69987/jacs.2024.40206
- [5] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Zican Dong, Yupeng Hou, Beichen Zhang, Yingqian Min, Junjie Zhang, Peiyu Liu, Xiaolei Wang, Yifan Du, Yushuo Chen, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Peiyu Liu, Yiwen Hu, Jian-Yun Nie, Ji-Rong Wen, "A Survey of Large Language Models", 2026, 10.1007/s11704-026-60308-3
- [6] Haoran Dong, "Tool-Augmented Large Language Model Agents for Analysis: A Focused Study", 2026, 10.2139/ssrn.6647800